

Rapport final – Projet E6 Data Engineering

ShopNow Marketplace

Olivier Kotwica

Data Ingénieur

Rapport final – Projet E6 Data Engineering

****ShopNow Marketplace – Architecture, exploitation et gouvernance****

Table des matières

1. Étape 1 – Cadrage de la mission et des livrables
2. Étape 2 – Analyse de l'existant
3. Étape 3 – Organisation de la maintenance
4. Étape 4 – Journalisation, supervision et alertes
5. Étape 5 – Sauvegardes complètes et partielles
6. Étape 6 – Intégration des sources Marketplace
7. Étape 7 – Sécurité, RGPD et gestion des accès
8. Étape 8 – Gestion des variations (SCD)
9. Étape 9 – Monitoring, documentation et rapport final
10. Conclusion générale et perspectives

Étape 1 – Cadrage de la mission et des livrables

(C16 / C17 – Cadrage et objectifs du projet)

1.1 Contexte

La plateforme **ShopNow** est initialement conçue selon un modèle de **e-commerce classique**, dans lequel l'entreprise gère directement l'ensemble du catalogue produits, des ventes et de la relation client.

Dans une logique d'évolution métier, ShopNow engage une **transition vers un modèle Marketplace**, permettant à **plusieurs vendeurs externes** de publier, gérer et vendre leurs propres produits via la plateforme.

Cette évolution implique des impacts majeurs sur le **système d'information décisionnel**, et en particulier sur le **Data Warehouse (DWH)** existant :

- multiplication des sources de données,
- hétérogénéité des flux entrants,
- besoin de suivi et d'analyse par vendeur,
- exigences accrues en matière de sécurité, traçabilité et qualité des données.

Le DWH doit donc être repensé pour accompagner cette transformation stratégique.

1.2 Objectifs globaux de la mission

La mission confiée consiste à **adapter et sécuriser l'architecture décisionnelle** afin de répondre aux nouveaux besoins induits par le modèle Marketplace.

Objectifs fonctionnels

- Adapter le **modèle de données du DWH** pour intégrer la notion de vendeur.
- Permettre des analyses multi-vendeurs (ventes, performance, volumes).
- Assurer une séparation logique des données dans un contexte multi-tenant.

Objectifs techniques

- Garantir la **qualité des données** issues de sources externes.
- Sécuriser l'accès aux données et aux infrastructures.
- Renforcer la **fiabilité** et la **résilience** de la plateforme.
- Mettre en place des mécanismes de **journalisation, supervision et sauvegarde**.

Objectifs méthodologiques (C16 / C17)

- Appliquer les bonnes pratiques de ****maintenance, supervision et exploitation**** (C16).
- Industrialiser les déploiements via ****Infrastructure as Code (Terraform)**** (C17).
- Structurer la documentation et les processus d'exploitation.

1.3 Livrables attendus

La mission donne lieu à la production des livrables suivants :

- ****Rapport professionnel**** détaillant :
 - l'analyse de l'existant,
 - les évolutions proposées,
 - les choix d'architecture et de maintenance***(volume attendu : 5 à 10 pages)*.**
- ****Présentation orale**** synthétique :
 - contexte,
 - problématique,
 - solution proposée,
 - démonstration des apports techniques***(durée : 5 à 10 minutes)*.**
- ****Documentation d'exploitation**** :
 - procédures de maintenance,
 - supervision et alertes,
 - sauvegardes et restauration,
 - rôles et responsabilités.

Ces livrables visent à démontrer la capacité à concevoir, exploiter et maintenir une plateforme décisionnelle conforme aux attentes professionnelles.

Étape 2 – Analyse de l'existant

(C16 – Analyse et compréhension de l'architecture)

2.1 Architecture DWH actuelle

L'architecture actuelle du Data Warehouse (DWH) de la plateforme **ShopNow** repose sur un modèle décisionnel classique en **étoile**, adapté à l'analyse des ventes et des comportements utilisateurs.

Modèle de données

Le schéma est composé des tables suivantes :

- **dim_customer** : informations clients (identité, localisation).
- **dim_product** : catalogue produits.
- **fact_order** : faits de vente (commandes, quantités, montants).
- **fact_clickstream** : événements de navigation utilisateurs.

Ce modèle permet des analyses orientées **clients, produits et ventes**, mais reste limité à un contexte mono-vendeur.

Ingestion et traitement des données

- **Ingestion en temps réel** via **Azure Event Hubs**, alimentés par des producteurs d'événements.
- **Traitement streaming** assuré par **Azure Stream Analytics**, transformant les événements entrants et alimentant le DWH.
- **Processus ETL planifiés** (quotidiens) pour les traitements complémentaires et consolidations.

Cette architecture garantit une faible latence entre la génération des événements et leur disponibilité analytique.

2.2 Limites actuelles identifiées

L'analyse de l'existant met en évidence plusieurs limites face aux nouveaux besoins de type **Marketplace**.

Limites fonctionnelles

- **Absence de la dimension vendeur** :
- Impossible d'analyser les ventes par vendeur.
- Pas de différenciation entre produits internes et vendeurs tiers.
- **Pas de cloisonnement multi-tenant** :

- Les données de tous les vendeurs sont agrégées sans séparation logique.
- Risque de non-conformité en cas d'accès différencié.

Limites techniques

- **Peu de mécanismes de qualité des données** pour les sources externes :
- Données incomplètes ou incohérentes possibles.
- Absence de contrôles automatiques à l'ingestion.
- **Processus ETL peu adaptés à des flux hétérogènes** :
- Difficulté à intégrer des formats ou fréquences variables.
- Faible scalabilité face à l'augmentation du nombre de vendeurs.

Ces limites justifient une évolution de l'architecture et du modèle de données.

2.3 Analyse de l'infrastructure existante

L'infrastructure actuelle a été analysée à partir de la configuration **Terraform** existante.

Ressources déjà en place

- **Azure Event Hubs** pour l'ingestion des flux temps réel.
- **Azure Stream Analytics** pour les traitements streaming.
- **Azure SQL Database** hébergeant le DWH.
- **Azure Log Analytics Workspace** pour la centralisation des logs et métriques.
- **Sauvegardes automatiques** (PITR et LTR) configurées sur la base SQL.

État des mécanismes transverses

- **Logs** :
- Disponibles pour Event Hubs, Stream Analytics et SQL.
- Centralisés dans Log Analytics.
- **Sauvegardes** :
- Sauvegardes automatiques actives.
- Rétention conforme aux besoins définis.
- **Sécurité et accès** :
- Accès administrateur SQL existant.
- Séparation des rôles encore limitée (amélioration possible).

Cette analyse confirme que l'infrastructure fournit une base solide, mais nécessite des adaptations pour répondre aux exigences d'une plateforme Marketplace multi-vendeurs.

Étape 3 – Organisation de la maintenance

(C16 – Méthodologie d'exploitation)

3.1 Modèle d'organisation de la maintenance (inspiré ITIL)

La maintenance de la plateforme **ShopNow Marketplace** s'appuie sur un modèle organisationnel inspiré des bonnes pratiques **ITIL (Information Technology Infrastructure Library)**, afin d'assurer la continuité de service, la traçabilité des actions et une amélioration continue.

Un **outil de ticketing** (ex. Jira, GLPI, Wrike) est utilisé pour centraliser l'ensemble des demandes et incidents. Chaque événement est formalisé sous forme de ticket, permettant un suivi structuré du cycle de vie.

Typologie des tickets

Les tickets sont classifiés selon leur nature :

- **Incident** : interruption ou dysfonctionnement non prévu du service (ex. arrêt du job Stream Analytics, indisponibilité du DWH).
- **Problème** : analyse approfondie d'incidents récurrents afin d'en identifier la cause racine.
- **Demande de service** : action standard planifiée (ex. création d'un accès, restauration d'une sauvegarde).
- **Évolution** : amélioration ou adaptation fonctionnelle ou technique (ex. ajout d'une nouvelle dimension dans le DWH).

Cette classification permet d'adapter les processus de traitement, les délais et les responsabilités associées.

3.2 Priorisation des incidents et SLA

Les tickets sont priorisés en fonction de leur **impact métier** et de leur **urgence**, avec des **engagements de temps de résolution (SLA)** clairement définis.

Priorité	Description	Exemple	SLA de résolution
-----	-----	-----	-----
P1 – Critique	Rupture totale de service	DWH indisponible, flux Event Hub arrêté	< 4 heures
P2 – Majeure	Dégradation significative	Retard de traitement, erreurs ASA	< 24 heures
P3 – Mineure	Faible impact ou demande	Évolution, optimisation	< 72 heures

Ce mécanisme garantit une allocation efficace des ressources et une réponse proportionnée à l'impact business.

3.3 Rôles et responsabilités

La maintenance repose sur une **répartition claire des rôles**, chacun disposant d'un périmètre de responsabilité défini.

Rôles clés

Data Engineer

- Supervision des pipelines de données (Event Hubs, Stream Analytics).
- Correction des erreurs de transformation et de qualité des données.
- Implémentation des évolutions du modèle de données.

Administrateur DWH

- Gestion du serveur SQL et du Data Warehouse.
- Supervision des performances, sauvegardes et restaurations.
- Application des politiques de sécurité et de rétention.

Exploitant / Support

- Surveillance quotidienne via les dashboards et alertes.
- Qualification et routage des tickets.
- Communication avec les équipes métier en cas d'incident.

Chaque rôle intervient selon des procédures documentées, garantissant la **traçabilité**, la **réversibilité des actions** et la **continuité opérationnelle**.

Étape 4 – Journalisation, supervision et alertes (C16)

4.1 Journalisation

L'objectif de la journalisation est de disposer d'une **traçabilité technique et fonctionnelle** de bout en bout sur la chaîne :

> Event producers → Event Hubs → Stream Analytics → Azure SQL DWH

4.1.1 Cible : centraliser les logs

Tous les logs sont centralisés dans un **Azure Log Analytics Workspace**, via les **Diagnostic Settings** :

- **Event Hubs**
 - Logs : erreurs de consommation, quota, throttling
 - Metrics : nombre de messages, latence, taille, etc.
- **Stream Analytics (asa-shopnow)**
 - Logs : erreurs de requête, erreurs d'output, restart du job
 - Metrics : nombre d'events traités, throughput, backlog
- **Azure SQL (dwh-shopnow)**
 - Logs : erreurs SQL, deadlocks, timeouts
 - Metrics : CPU/DTU, taille, connexions, I/O
- **Conteneur aeh-producers**
 - Logs applicatifs (stdout / stderr) via Container Instances / Container Apps

4.1.2 Logs techniques vs logs fonctionnels

- **Logs techniques**
 - erreurs Event Hub (send/receive),
 - erreurs ASA (outputs, parsing),
 - erreurs SQL (timeouts, connexion).
- **Logs fonctionnels**
 - anomalies dans les données (`seller\_id` manquant),
 - incohérences produit/vendeur,
 - contrôles de qualité sur le DWH.

4.1.3 Structure des logs (convention)

Format recommandé pour les logs applicatifs :

```
```json
{
 "timestamp": "2025-11-26T09:15:23Z",
 "source": "producer",
 "type": "TECH",
 "level": "ERROR",
 "message": "Failed to send order event",
 "error_code": "EH_SEND_FAIL",
 "seller_id": "f5b0e8d3-...",
 "order_id": "a3b1c2d4-..."
}
```
```

Champs standard :

- `timestamp`,
- `source`,
- `type` (TECH/FUNC),
- `level` (INFO/WARN/ERROR),
- `message`,
- `error\_code`,
- `seller\_id` (si applicable).

4.2 Catégorisation des logs

4.2.1 Niveaux de sévérité

- **INFO** : fonctionnement normal
- **WARNING** : anomalie non bloquante
- **ERROR** : incident critique / perte potentielle de données

4.2.2 Typologie par domaine

- **Sécurité** : connexions SQL échouées, rôle modifié
- **Qualité** : anomalies dans les données, incohérences
- **Exploitation** : erreurs ASA, backlog Event Hub, crash du container producers

4.3 Alertes automatiques

Les alertes reposent sur **Azure Monitor** et Log Analytics.

4.3.1 Cas d'alerte principaux

1. **Échec du job Stream Analytics**
 - ASA passe en `Failed` ou `Stopped`
 - Envoi e-mail + notification Teams
2. **Anomalies vendeur**
 - taux d'erreurs pour un vendeur > seuil
 - données invalides dans `fact\_order`
3. **SLA DWH non respecté**
 - latence entre Event Hub et DWH > 5 min
4. **Anomalies Event Hub**
 - 0 messages reçus pendant 10 minutes
 - backlog anormalement élevé

4.3.2 Canaux d'alerte

- E-mail (default)
- Teams (critique)
- SMS (optionnel / production)

4.4 Tableau de bord de supervision

4.4.1 Indicateurs techniques

- **Taux de succès ETL**
- **Statut ASA** (Running / Failed)
- **Latence DWH**
- **Santé SQL** : CPU, DTU, connexions

4.4.2 Indicateurs fonctionnels

- Volume par vendeur
- Taux d'erreurs vendeur
- Qualité des données (`seller\_id` null, `unit\_price` incohérent)

4.4.3 Outils possibles

- **Azure Dashboard** (temps réel)
- **Log Analytics Workbooks** (KQL visuel)
- **Power BI** (préconisation cible)

Conclusion

Cette étape met en place les fondations d'une supervision complète et alignée avec les attentes de la compétence C16 : visibilité, réactivité, maîtrise de la qualité et de la sécurité du système.

Étape 5 – Sauvegardes complètes et partielles

5.1 Ce qui doit être sauvegardé

Dans l'architecture ShopNow Marketplace, plusieurs éléments critiques nécessitent une stratégie de sauvegarde structurée :

1. Base SQL du DWH

Elle contient :

- dim_customer
- dim_product
- dim_seller
- fact_order
- fact_clickstream

C'est la source de vérité analytique du système.

2. Schémas & métadonnées DWH / ETL

À sauvegarder :

- Le script SQL `dwh\_schema.sql`
- La requête Stream Analytics (transformation_query)
- Les fichiers ETL / code producteurs (`producers.py`)
- La configuration Terraform (notamment stream_analytics/main.tf)

3. Zone de staging

Event Hub constitue la zone de transit des événements.

La rétention native peut servir de sauvegarde minimale mais doit être complétée par **Event Hub Capture** si besoin d'archivage long terme.

5.2 Types de sauvegarde

■ 1. Sauvegarde complète du DWH

- **Fréquence recommandée : quotidienne (nuit).**
- Méthode : Azure SQL Automatic Backups.
- Conservation : 7 à 35 jours selon le tier.

■ 2. Sauvegarde partielle (tables critiques)

Les tables les plus sensibles :

- `fact\_order` (transactions)

- `dim\_seller` (nouveau modèle Marketplace)
- `dim\_product` (relation produit-vendeur)

Exemple SQL simple :

```
```sql
SELECT * INTO backup.fact_order_20250201
FROM fact_order;
```
```

Ou export automatisé via `bcp` :

```
```bash
bcp "SELECT * FROM dim_seller" queryout dim_seller_20250201.csv -S server -d db -U user -P
pass -c
```
```

■ 3. Sauvegarde des métadonnées

Comprend :

- `dwh\_schema.sql`
- `main.tf` du job Stream Analytics
- `producers.py`
- Scripts SQL de création de tables, vues, clés étrangères

Bonne pratique :

- Versionner l'ensemble dans GitHub.
- Snapshot hebdo automatique.

■ 4. Sauvegarde Event Hub (staging)

- Rétention 1–7 jours → backup minimal
- Option recommandée : ****Event Hub Capture vers Azure Blob Storage****
- Permet de conserver un historique long terme du flux brut

5.3 Procédures de restauration

■ Restauration complète

1. Aller dans Azure SQL → Backups.
2. Choisir le point dans le temps.
3. Restaurer la base (overwrite ou nouvelle base).
4. Vérifier l'intégrité (COUNT(*), clés étrangères).
5. Redémarrer le job Stream Analytics et le conteneur `aeh-producers` si nécessaire.

■ Restauration partielle d'une table

Exemple : `dim\_seller`

1. Créer une table temporaire :

```
```sql
SELECT * INTO dim_seller_restore
FROM backup.dim_seller_20250201;
```
```

2. Vérifier la cohérence.

3. Effectuer la restauration :

```
```sql
DELETE FROM dim_seller;
INSERT INTO dim_seller SELECT * FROM dim_seller_restore;
```
```

Exemple : `fact\_order`

1. Sauvegarde préalable :

```
```sql
SELECT * INTO fact_order_before_restore FROM fact_order;
```
```

2. Restauration :

```
```sql
DELETE FROM fact_order;
INSERT INTO fact_order SELECT * FROM backup.fact_order_20250201;
```
```

■ Restauration des métadonnées

Stream Analytics

- Réappliquer le Terraform :

```
```bash
terraform apply
```
```

→ Recrée les outputs, inputs et transformation_query.

Schéma SQL

- Exécuter `dwh\_schema.sql` pour reconstruire la structure en cas d'erreur critique.

5.4 Tests périodiques de restauration (C16)

Test mensuel

1. Restaurer un full backup dans une base `dwh\_restore\_test`.
2. Vérifier :
 - volumes de données,
 - cohérence des dimensions,
 - qualité des relations vendeur-produit.

Test trimestriel

- Restaurer un datamart (exemple : `dim\_product`)
- Vérifier l'intégrité référentielle avec `dim\_seller`

Résumé

Cette stratégie garantit :

- la sécurité des données,
- la disponibilité opérationnelle,
- la conformité Marketplace,
- la traçabilité complète,
- et une résilience maîtrisée, conformément aux compétences C16 / C17.

Étape 6 – Intégration des nouvelles sources Marketplace

(C16 / C17 – Intégration, qualité et évolutivité des données)

6.1 Zone de staging / ingestion

Afin d'intégrer efficacement les nouvelles sources de données issues des vendeurs externes, une **zone de staging** est mise en place.

Cette zone constitue un espace tampon entre les sources Marketplace et le Data Warehouse.

Rôles de la zone de staging

- Centraliser les données brutes provenant :
 - de fichiers (CSV, JSON),
 - d'API exposées par les vendeurs,
 - de flux événementiels.
- Appliquer une **normalisation initiale** des formats (encodage, structure, typage).
- Isoler les données non conformes avant leur propagation dans le DWH.

Contrôles automatiques à l'ingestion

- Vérification du **format des fichiers** (JSON valide, schéma attendu).
- Contrôle de la **structure des messages** (présence des champs obligatoires).
- Rejet immédiat des données illisibles ou corrompues.

6.2 Nouvelle dimension : `dim\_seller`

Dans un contexte Marketplace, l'introduction de la dimension **vendeur** est indispensable pour assurer le suivi et l'analyse multi-tenant.

Champs recommandés

La table `dim\_seller` contient notamment :

- `seller\_id` : identifiant unique du vendeur.
- `seller\_name` : nom commercial.
- `country` : pays d'origine.
- `category` : catégorie principale du vendeur.
- `status` : état du vendeur (actif, suspendu, désactivé).
- `created\_at`, `updated\_at` : dates de création et de mise à jour.

Selon les besoins analytiques, une gestion **SCD (Slowly Changing Dimension)** peut être mise en place afin de conserver l'historique des évolutions (changement de statut, de catégorie, etc.).

6.3 Qualité des données

L'ouverture du SI à des sources externes impose des **règles strictes de qualité des données**.

Contrôles de qualité appliqués

- **Présence des champs obligatoires** (`seller\_id`, `product\_id`, etc.).
- **Format des identifiants** (UUID, longueurs, unicité).
- **Validation des catégories produits** par rapport à une liste de référence.
- **Détection des doublons** (vendeurs ou produits).

Traitement des anomalies

En cas de non-conformité :

- Rejet des enregistrements en **table d'erreurs** dédiée.
- Tentative de **nettoyage automatique** si possible (trim, cast, mapping).
- **Retour au fournisseur** avec description de l'erreur pour correction.

Ces mécanismes permettent de préserver la fiabilité du DWH.

6.4 Flux ETL Marketplace

Le flux ETL d'intégration des vendeurs suit une chaîne de traitement standardisée :

1. **Ingestion** des données dans la zone de staging.
2. **Contrôles qualité** et validation des formats.
3. **Normalisation** des données (mapping des champs, harmonisation).
4. **Chargement des dimensions** (`dim\_seller`, `dim\_product`).
5. **Chargement des tables de faits** (`fact\_order`, autres faits Marketplace).
6. **Mise à jour des logs et du monitoring** (statuts, volumes, erreurs).

Ce pipeline garantit une intégration contrôlée, traçable et scalable.

6.5 Procédure : “Ajouter une nouvelle source vendeur”

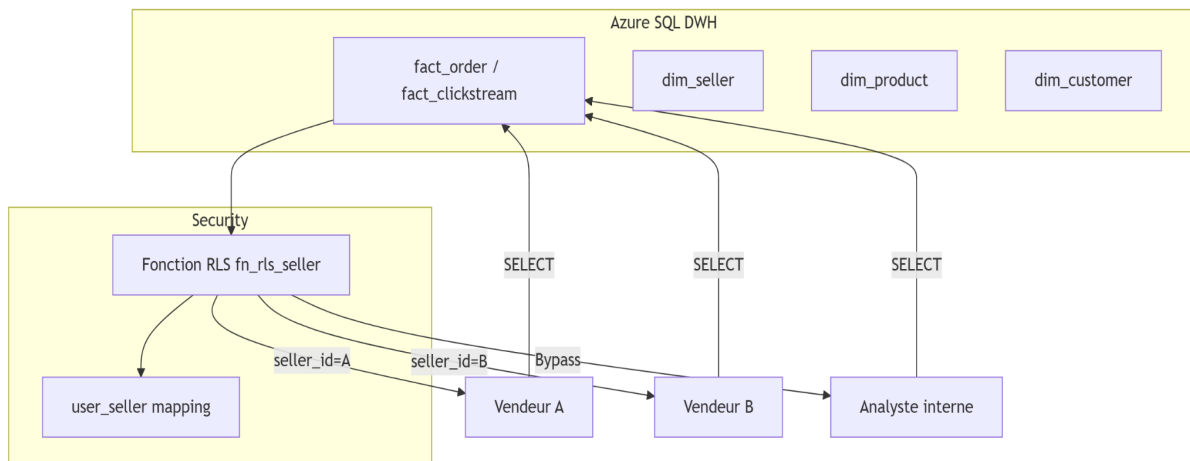
Une procédure standardisée est définie afin d'assurer une intégration sécurisée et reproductible.

Étapes de la procédure

1. ****Collecte des informations de mapping**** :
 - structure des données,
 - correspondance des champs,
 - règles spécifiques du vendeur.
2. ****Tests de qualité**** sur un jeu de données pilote.
3. ****Activation de la source**** dans les flux d'ingestion.
4. ****Vérification post-production**** :
 - contrôle des volumes,
 - vérification des indicateurs qualité,
 - validation des données dans le DWH.

Cette procédure permet une montée en charge progressive et maîtrisée du nombre de vendeurs intégrés.

Étape 7 – Sécurité, RGPD et gestion des accès



(C16 / C17 – Sécurité, conformité et gouvernance des données)

7.1 Cloisonnement multi-vendeurs

Dans un contexte **Marketplace multi-vendeurs**, la sécurité des données repose sur un cloisonnement strict afin de garantir que chaque acteur n'accède qu'aux informations qui le concernent.

Séparation logique des données

- Mise en place de **Row-Level Security (RLS)** au niveau du Data Warehouse, basée sur le champ `seller_id`.
- Les règles de sécurité sont appliquées directement au niveau de la base de données, indépendamment des outils de restitution.
- Chaque requête est automatiquement filtrée selon l'identité et le rôle de l'utilisateur.

Accès différencié selon les usages

- Les **vendeurs** n'accèdent qu'aux données associées à leur `seller_id`.
- Les **équipes internes** disposent d'une vision consolidée et transverse de la Marketplace.
- Des **vues filtrées et sécurisées** sont utilisées pour exposer uniquement les données nécessaires selon les profils.

Ce mécanisme garantit :

- la confidentialité des données inter-vendeurs,
- la conformité contractuelle,
- une isolation forte sans duplication physique des données.

7.2 Rôles d'accès et gestion des permissions

La gestion des accès repose sur une définition claire des **rôles utilisateurs**, chacun disposant de permissions adaptées à ses responsabilités.

Rôles définis

Administrateur DWH

- Accès complet aux données et aux structures.
- Gestion des utilisateurs, rôles et politiques de sécurité.
- Supervision des performances, sauvegardes, restaurations et journaux de sécurité.

Analyste interne

- Accès aux données agrégées et indicateurs métier.
- Pas d'accès direct aux données personnelles complètes (PII).
- Utilisation de **vues anonymisées ou pseudonymisées** pour l'analyse.

Vendeur

- Accès strictement limité aux données liées à son `seller_id`.
- Consultation des ventes, performances et indicateurs le concernant.
- Aucun accès aux données clients non nécessaires à son activité.

Cette organisation applique le **principe du moindre privilège** et réduit les risques d'exposition de données sensibles.

7.3 Conformité RGPD

La plateforme intègre des mécanismes de conformité au **Règlement Général sur la Protection des Données (RGPD)** afin de garantir une exploitation responsable des données personnelles.

7.3.1 Minimisation et contrôle des données personnelles

- Les données personnelles sont **centralisées** dans la dimension `dim_customer`.
- Les tables de faits utilisent des **identifiants techniques**.

- Les vendeurs n'ont aucun accès direct aux données clients identifiantes.
- Les analystes internes travaillent principalement sur des ****données agrégées ou anonymisées****.

7.3.2 Politique de conservation des données

- Données de navigation : conservation courte.
- Données transactionnelles : conservation longue pour obligations légales.
- Données agrégées : conservation étendue.

Une ****purge automatique planifiée**** est mise en place pour supprimer les données obsolètes.

7.3.3 Droit à l'oubli et anonymisation

- Anonymisation des données personnelles.
- Conservation des faits métiers.
- Utilisation d'un identifiant client neutre ('ANON').

Une procédure dédiée permet de traiter les demandes RGPD de manière sécurisée et traçable.

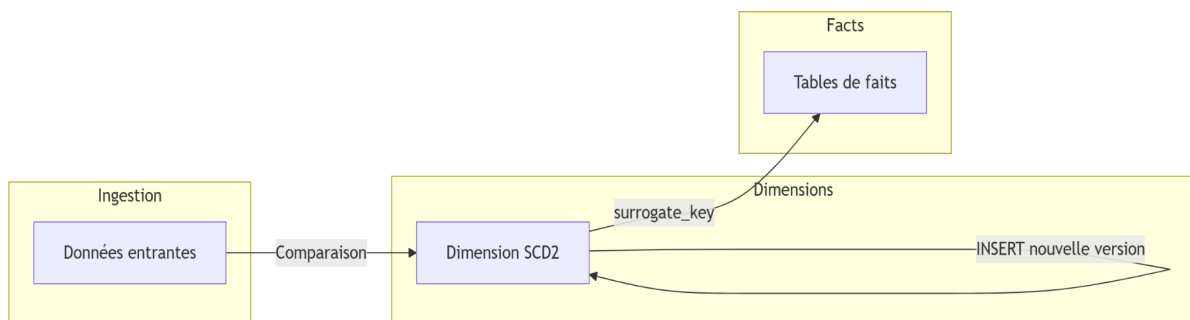
7.3.4 Traçabilité et gouvernance

- Journalisation des accès et actions sensibles.
- Suivi des opérations RGPD.
- Mise à jour du registre des traitements.

Conclusion

L'architecture garantit une exploitation ****sécurisée, conforme et gouvernée**** des données Marketplace, en accord avec les exigences réglementaires et les bonnes pratiques de l'ingénierie des données.

Étape 8 – Gestion des variations (SCD – C17)



Objectif : gérer l'évolution des dimensions (client / produit / vendeur) dans le temps sans perdre l'historique analytique, tout en restant compatible avec la sécurité multi-vendeur (RLS) et le RGPD.

8.1 Dimensions concernées

- **Customer** : évolutions d'adresse / localisation, corrections de contact.
- **Product** : changements de catégorie, renommage, rattachement vendeur.
- **Seller** : statut (actif/suspendu), catégorie, pays.

> Rappel : les **tables de faits** ne sont pas historisées en SCD. Elles doivent pointer vers une **version** de dimension (via une clé technique "surrogate key") afin de conserver le contexte historique.

8.2 Types de variations retenus

SCD Type 1 (écrasement)

- Utilisé pour les attributs **non historiques** (corrections, informations sans enjeu d'historisation).

- Implémentation : `UPDATE` de la version courante.

SCD Type 2 (historisation complète)

- Utilisé quand l'historique est nécessaire (pilotage, conformité, analyse temporelle).

- Implémentation :

1) clôturer la version courante (`valid\_to`, `is\_current = 0`)

2) insérer une nouvelle version (`valid\_from`, `valid\_to`, `is\_current = 1`)

SCD Type 3 (rare)

- Conservé uniquement en documentation (non implémenté ici) : une colonne "ancienne valeur".

8.3 Modélisation DWH (SCD2)

8.3.1 Colonnes standard SCD2

Pour chaque dimension historisée :

- `\_sk` : **surrogate key** (IDENTITY) – clé technique utilisée dans les faits
- `\_id` : **natural key** (identifiant métier) – stable
- `valid\_from` / `valid\_to` : période de validité
- `is\_current` : version courante

8.3.2 Conséquences sur les faits

Les faits doivent pointer sur la **version** de dimension :

- `fact\_order` : `customer\_sk`, `product\_sk`, `seller\_sk`
- `fact\_clickstream` : selon disponibilité, `seller\_id` peut rester nullable

■ **Compatibilité RLS** : on conserve aussi `seller\_id` (natural key) dans les tables où on applique le filtre RLS, ou on le conserve dans les dimensions filtrées.

8.4 Adaptation ETL (logique de traitement)

8.4.1 Principe général

- 1) Charger les données brutes en **staging** (ou flux stream → staging logique).
- 2) Comparer la "nouvelle" ligne entrante à la **version courante** (`is\_current = 1`) de la dimension.
- 3) Appliquer les règles SCD :
 - **SCD1** : mise à jour sur la ligne courante
 - **SCD2** : clôture + insertion nouvelle version

8.4.2 Pseudo-algorithme SCD2

Pour une clé métier `X\_id` :

- Si aucune ligne courante n'existe → `INSERT` (nouvelle version)
- Si existe et attribut(s) SCD2 différent(s) →
- `UPDATE` ligne courante : `valid\_to = now`, `is\_current = 0`
- `INSERT` nouvelle ligne : `valid\_from = now`, `valid\_to = '9999-12-31'`, `is\_current = 1`
- Sinon → rien à faire (idempotent)

8.4.3 Gestion des clés dans les faits

Au chargement des faits :

- Résoudre les `\*\_sk` en joignant sur la dimension (clé métier + timestamp) :
- version valide à la date de l'événement (entre `valid\_from` et `valid\_to`)

> Pour un premier niveau (E6), on peut choisir :

- > **(A)** "version courante au moment du chargement" (plus simple)
- > ou **(B)** "version valide à l'instant T" (meilleure historisation).

8.5 Documentation (tableau SCD)

| Dimension | Attribut | Type SCD | Justification |

|---|---|---|---|

| customer | name | SCD1 | correction / pas d'enjeu historique |

| customer | email | SCD1 | contact non utilisé pour analyse historique |

| customer | address / city / country | SCD2 | analyses géographiques dans le temps |

| product | name | SCD1 | renommage/correction |

| product | category | SCD2 | analyses "vente par catégorie" dans le temps |

| product | seller_id (changement de vendeur) | SCD2 | change le propriétaire du produit |

| seller | name | SCD1 | correction / libellé |

| seller | country / category | SCD2 | segmentation & reporting historique |

| seller | status | SCD2 | conformité / pilotage (suspension, réactivation) |

Mise en œuvre dans le projet (résumé)

- Créer / mettre à jour le schéma SQL vers SCD2 (voir fichier `dwh\_schema\_scd2.sql`).
- Mettre à jour les pipelines d'ingestion/ETL (Stream Analytics ou batch) pour :
- alimenter les dimensions via logique SCD
- charger les faits avec les `\*\_sk` (ou au minimum stocker `seller\_id` pour RLS)

- Tester :
- changement d'adresse client → nouvelle version dim_customer
- changement de statut vendeur → nouvelle version dim_seller
- vérifier les faits avant/après (historique cohérent)

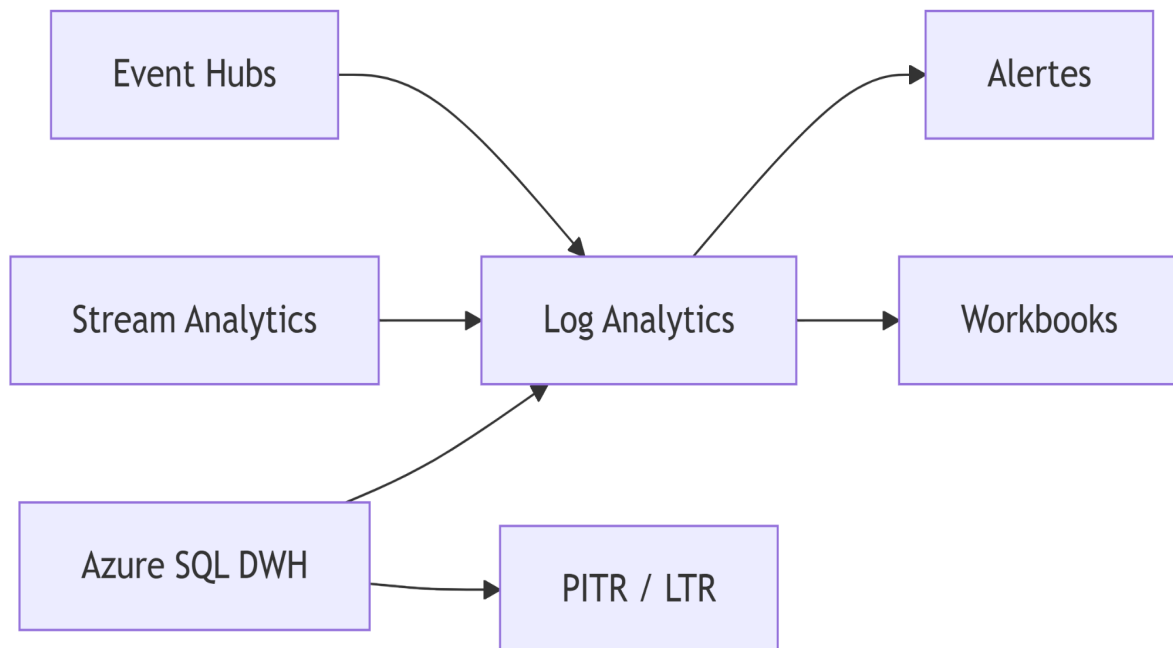
Contrôles attendus (qualité / audit)

- Unicité d'une seule ligne `is\_current = 1` par clé métier
- `valid\_from < valid\_to`
- `valid\_to = '9999-12-31'` pour version courante (convention)
- Logs ETL : nombre d'INSERT / UPDATE SCD1 / UPDATE+INSERT SCD2

Impacts sur sécurité (RLS) et RGPD (résumé)

- **RLS** : filtrer sur `seller\_id` (clé métier) pour rester stable malgré SCD2.
- **RGPD** : droit à l'oubli = anonymisation de `dim\_customer` (toutes versions) + éventuelle neutralisation des faits (selon politique).

Étape 9 – Monitoring, documentation et rapport final



(C16 / C17 – Exploitation, supervision et valorisation du projet)

9.1 Indicateurs de monitoring

La supervision du Data Warehouse Marketplace repose sur un ensemble d'indicateurs permettant de garantir la **fiabilité**, la **performance** et la **qualité des données**.

Indicateurs techniques (ETL & flux)

- **Succès / échec des traitements ETL**
- Surveillance des erreurs Stream Analytics.
- Détection des interruptions de flux Event Hubs.
- **Temps d'exécution et latence**
- Délai entre la réception d'un événement et son intégration dans le DWH.
- **Volumétrie**
- Nombre de messages entrants par Event Hub.
- Évolution de l'activité Marketplace.

Indicateurs qualité & Marketplace

- **Taux d'erreurs par vendeur**
- Données rejetées pour non-conformité.
- Erreurs de format ou de schéma.
- **Charges par vendeur**
- Volume de commandes.
- Activité clickstream par vendeur.

Indicateurs sécurité & fiabilité

- **État des sauvegardes**
- Sauvegardes automatiques (PITR).
- Sauvegardes long terme (LTR).
- **Alertes critiques**
- CPU SQL élevé.
- Absence de messages entrants.
- Erreurs Stream Analytics.

Ces indicateurs sont centralisés dans **Azure Monitor / Log Analytics** et visualisés via des **Workbooks**, avec déclenchement d'alertes automatiques.

9.2 Documentation d'exploitation

Une documentation d'exploitation est mise en place afin de garantir la **maintenabilité** et la **reproductibilité** du système.

Procédures principales

Ajouter un vendeur

1. Créer l'entrée vendeur dans la dimension `dim\_seller`.
2. Associer un `seller\_id` unique.
3. Mettre à jour le mapping de sécurité (RLS).
4. Vérifier l'accès et les données visibles.

Ajouter un accès utilisateur

- Attribution du rôle approprié (`seller\_role`, `internal\_analyst`, `dwh\_admin`).
- Vérification des permissions.
- Test du cloisonnement via RLS.

Restaurer une sauvegarde

- Choix du type de restauration (PITR ou LTR).

- Restauration dans une base temporaire.
- Validation de l'intégrité.
- Remplacement de la base si nécessaire.

Ajouter un datamart

- Identification du besoin métier.
- Création de vues ou tables agrégées.
- Sécurisation par rôles et RLS.
- Ajout au monitoring et aux dashboards.

Vérifier les logs et alertes

- Consultation du Log Analytics Workspace.
- Analyse de l'historique des alertes.
- Vérification des diagnostics et métriques.

9.3 Rapport professionnel

Le rapport final vise à présenter le projet de manière claire, structurée et professionnelle.

Plan recommandé

1. Contexte et objectifs
2. Problématique de la transformation Marketplace
3. Analyse de l'existant
4. Organisation de la maintenance
5. Journalisation, monitoring et alertes
6. Sauvegardes et restauration
7. Intégration des vendeurs Marketplace
8. Sécurité, RGPD et gestion des accès
9. Gestion des évolutions (SCD & ETL)
10. Conclusion et perspectives

Chaque section présente :

- le problème identifié,
- la solution mise en œuvre,
- la justification des choix techniques.

9.4 Préparation orale

Pitch (5–7 minutes)

- **Introduction** : contexte ShopNow et évolution vers une Marketplace.
- **Problématique** : multi-vendeurs, sécurité, scalabilité, gouvernance.
- **Architecture cible** : Event Hubs → Stream Analytics → DWH.
- **Points forts techniques** :
 - Cloisonnement multi-vendeurs (RLS).
 - Gestion des évolutions (SCD2).
 - Sauvegardes et restauration.
 - Monitoring et alertes.
- **Gouvernance & conformité** :
 - RGPD.
 - Droit à l'oubli.
 - Gestion des accès.
- **Conclusion** : solution robuste, évolutive et conforme.

Questions types à anticiper

- Pourquoi privilégier l'anonymisation plutôt que la suppression des faits ?
- Pourquoi utiliser des SCD de type 2 ?
- Comment garantir qu'un vendeur n'accède qu'à ses données ?
- Que se passe-t-il en cas de panne ou d'erreur critique ?

Conclusion – Étape 9

Cette étape permet de valoriser l'ensemble du projet en démontrant :

- une vision **exploitation et long terme**,
- une **maîtrise du monitoring et de la supervision**,
- une capacité à documenter et transmettre,
- une préparation solide à l'oral.

Elle confirme que la solution mise en place est **opérationnelle, maintenable et conforme**, répondant pleinement aux exigences des compétences **C16 et C17**.

Conclusion générale et perspectives

Le projet **ShopNow Marketplace** a permis de transformer une architecture décisionnelle classique en une **plateforme Data moderne, sécurisée et gouvernée**, adaptée à un contexte multi-vendeurs.

Les principaux objectifs ont été atteints :

- adaptation du Data Warehouse au modèle Marketplace,
- intégration contrôlée de sources externes hétérogènes,
- cloisonnement strict des données par vendeur,
- conformité aux exigences RGPD,
- mise en place d'un monitoring, de sauvegardes et de procédures d'exploitation robustes.

L'approche **Infrastructure as Code** via Terraform, combinée aux services managés Azure, garantit la reproductibilité, la résilience et la maintenabilité de la solution.

Perspectives d'évolution

Plusieurs évolutions peuvent être envisagées :

- ajout d'une couche de restitution avancée (Power BI),
- automatisation complète des scénarios de restauration,
- renforcement des contrôles de qualité des données,
- intégration de cas d'usage Data Science,
- montée en charge vers un environnement de production à grande échelle.

Ce projet démontre une maîtrise complète du cycle de vie des données et répond pleinement aux exigences des compétences **C16 et C17**.
